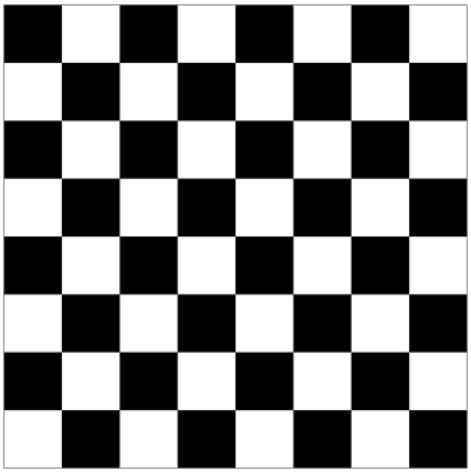
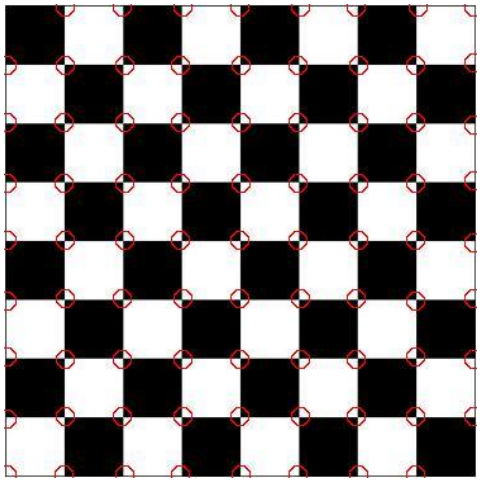
APS106 – Lab #7

This week you will create functions to perform some simple, yet powerful, operations for image processing. In the process of this lab, you will build a simplified corner detector that will be able to analyze images and return the location of potential "corners" present within the image.

Input Image	Identified corners indicated by red circles
	

As usual, we will not be concerned with any complex theories or math behind these approaches but focus on how to implement these algorithms using the tools you have learned in APS106. The focus of these exercises is to practice *using lists, tuples, and loops* with structured data. The lab may seem a little overwhelming in places because we are giving you background information about the problem, but we have broken the problem down into small functions that are well within your capabilities as APS106 students.

Take time to read the instructions carefully, give yourself time to complete the lab, and ask questions in your tutorials and practicals when you get stuck. If you think carefully about each function, **you can finish the lab by writing less than 70 lines of code**. Most importantly, **have fun**. This lab is designed to show you some of the really cool things you can do with only a little bit of programming experience!

Lab Objectives

- ☐ Use data containers (list, tuples, dictionaries) to represent and manipulate structured data
- ☐ Extract data from data containers using indexing, slicing, and nested loops

Lab Deliverables

The following files must be submitted to Gradescope prior to the assignment deadline:

- ☐ `lab7.py`

Functions to implement for this lab:

- `convert_rgb_to_gs`
- `dot_product`
- `extract_2D_img_segment`
- `kernel_filter`
- `non_maxima_suppression`

Two additional functions are provided within the starter code. You may use these functions to complete and test your code as needed. **You are not expected to modify these two functions**.

- `harris_corner_strength`
- `harris_corners`

A Note on the Gradescope autograder. Your assignment will be graded using 10 test cases on Gradescope. You will be able to see the results of all of these tests before the deadline. However, you will only be able to see the inputs of the first **five** test cases before the submission deadline. If you are not passing any of the test cases, there is an error (bug) in your submission that you will need to identify and correct.

IMPORTANT:

- Do not change the file, function, class, or method names
- Do not use `input()` inside your program
- **Using numpy, scipy, opencv or other image processing packages to complete this lab is strictly prohibited and will result in a grade of zero.**

Introduction (Important information, read carefully)

For this week's lab, you will imagine you are working on a team developing autonomous vehicles. As a new engineer on the team, you have been asked to create some tools to detect and track the movement of objects in the vehicle's environment using the video from cameras in the vehicle's sensor system. As a starting point, someone tells you that corner detection can be used as a simple method for object movement detection and tracking. So, you set out, armed with your APS106 programming skills, to tackle this problem…

At this point you may be a little confused. We have not discussed "images" as a data type within the lectures and many of you may be asking questions like:

- What do images look like when stored in a variable?
- How do I get an "image" into my program?

Luckily, both questions are easy to answer using things we have seen in this course!

Representing images as lists or tuples of numbers

Computers store all information in binary code, so they "see" images quite a bit differently than us humans. Rather than seeing abstract elements like colours, shapes, and objects, computers see a grid of numbers. These numbers are referred to as "pixels." The colour or intensity of an image at a location on the image is determined by the value of the pixel at that location (figure 1). Check out this short 2-minute video for a quick overview of how computers see images: https://realpython.com/lessons/how-computers-see-images/.

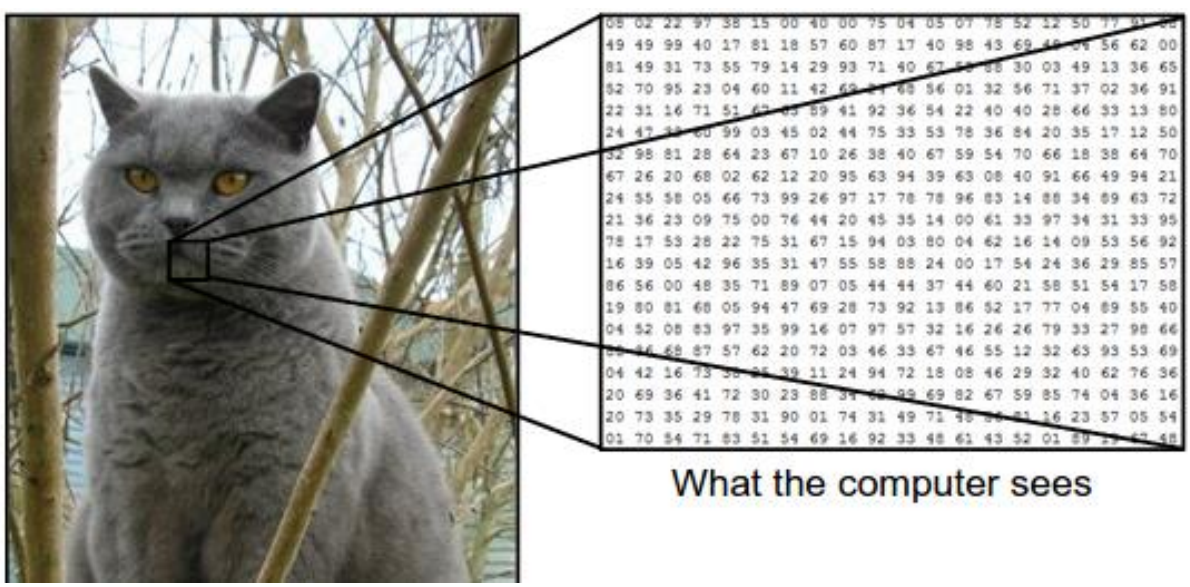


Figure 1. Computers represent images as a two-dimensional grid of pixel values. Image source: https://cs231n.github.io/

Within a computer program, an image is simply an *ordered* collection of numbers. **This means we can store images in python as lists or tuples of integers!** For example, we could represent an image using the following tuple of numbers:

```
pixels = (250, 253, 255, 223, 181, 184, 232, 255,  ...
```

Each one of these numbers is the value of a pixel at a particular location in the image. Pixel values can be individual numbers for black and white images or a tuple of three values for colour images (more details provided in Part 1 – RGB to grayscale).

How 2-dimensional images are stored in 1-dimensional lists/tuples

Pixel locations are described as 2-dimensional coordinates along the height and width of the image. By convention, the **top left** corner pixel is the coordinate (0,0). Increasing the x-coordinate is equivalent to moving from left to right across the image. Increasing the y-coordinate is equivalent to moving from the top to the bottom of the image (figure 2).

Figure 2. Coordinate axis conventions for images. Note that increasing the y-coordinate results in moving towards the bottom of the image.

For our program this week, you will be storing image pixels in a **1-dimensional tuple**. Pixels will be ordered row-by-row (also referred to as **row-major order**). This means the pixel at coordinate (0,0) (the top left corner pixel) is stored in list index 0, the pixel at coordinate (x=1, y=0) (the second pixel from the left on the top row) is stored at index 1, the pixel at coordinate (x=2, y=0) is stored at index 2, and so on. Once we reach the end of a row, we move down a row (increase y) and start again from the left of the image. Figure 3 illustrates how a 3x3 (three rows, three columns) image would be represented within a tuple.

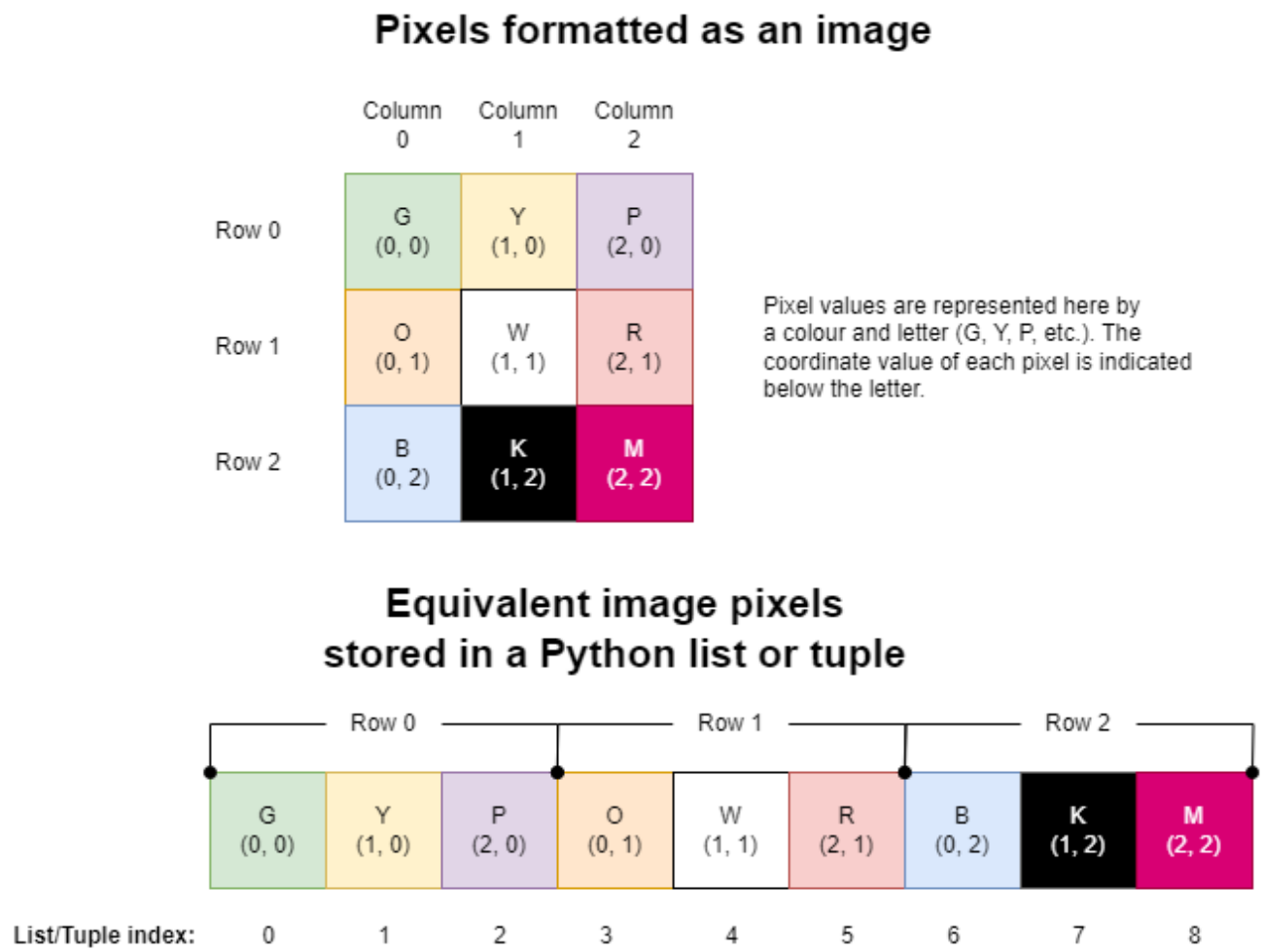


Figure 3. Visual representation of how images can be stored in tuples or lists. Pixels within the same row are stored together in adjacent tuple/list indices.

For this lab, you will need to devise a method to calculate the one-dimensional tuple index of a pixel given the two-dimensional coordinate and the dimensions (height and width) of an image. Take a minute using the example in figure 3 to think about how you can do this.

Loading images into programs

To write and test the functions in this lab, we will simply define tuples of pixel values (we'll have examples below). These tuples won't represent real images, but they will be sufficient for writing and testing our code. If you want to try out the functions you will write with real images, refer to the notebook "optional_lab7_with_real_images.ipynb" available on Quercus.

Part 1 – RGB to Grayscale Conversion

The `convert_rgb_to_gs` function that converts a RGB (colour) image into grayscale (black and white). The input to the function is a tuple of RGB pixels and the function returns a tuple of corresponding grayscale pixels.

RGB Image	Grayscale Image

The image on the left is in RGB format. In this format, each pixel is a combination of different intensities of red, green, and blue pixels. RGB images are represented by *nested* tuples where each pixel value is a three-element tuple. The first element is the red intensity, the second element is the green intensity, and the third element is the blue intensity.

```
pixel_tuple = ((R0, G0, B0), (R1, G1, B1), (R2, G2, B2), ...
```

Grayscale images, on the other hand, are made up of only one value per pixel. An RGB pixel can be converted to a grayscale pixel using the following equation:

$$Grayscale\ value = 0.3\,R + 0.59G + 0.11B$$

where R, G, and B are the red, green, and blue pixel intensities of the RGB pixel, respectively. All grayscale pixels computed by this function should be **rounded to the nearest integer**.

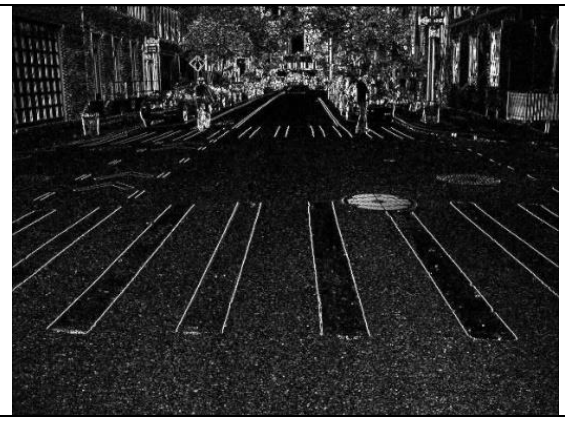
Sample Test Case

Input	Expected Output
`((3,67,90), (249, 255, 0), (49, 150, 128))`	`(50, 225, 117)`

Part 2 – Kernel Filtering

In this part of the lab, you will implement a function to apply filters to *grayscale images*. This filtering approach is called *convolutional filtering*, and it is the building block of many modern

deep learning and artificial intelligence algorithms. By the end of this part, your program will be able to produce the example image transformations shown below.

Original Image	Blurred
Sharpened	**Vertical Edge Detection**
	

The math behind these filters is beyond the scope of APS106; instead, the exercises here will focus on how to implement these filters using simple data containers and looping tools. In the next part, we will walk through the algorithm you need to implement. Then you will write three functions to perform these filtering operations.

Part 2.1 Problem Description (important information, read carefully)

Each individual pixel value in the output image is a weighted sum of pixels from a two-dimensional $N \times N$ (N pixels high & N pixels wide) segment of the input image. The weights for this sum can be defined and visualized using a 2-dimensional grid, referred to as a kernel.

As an example, a 3×3 "Gaussian blurring" kernel has the following weights:

1/16	1/8	1/16
1/8	1/4	1/8
1/16	1/8	1/16

To illustrate the filtering procedure, we will examine an example below using this kernel. The pixel values at each *(x,y)* coordinate of the *output* image are computed using the following four-step procedure:

1. Extract the $N \times N$ image segment (group of pixels) from the input image centred around the position *(x,y).* In this example, $N = 3$.
2. Multiply each pixel value from the $N \times N$ segment with the kernel weight in the corresponding location.
3. Sum the products from step 2.
4. Truncate or *round down* the sum from step 3 to the nearest integer.

We will illustrate this procedure with the following 7×7 input image example. Each number in the table represents a pixel value.

4	87	233	245	227	209	190
2	59	235	246	229	219	200
17	99	230	220	211	210	201
46	58	196	165	201	179	150
82	63	41	169	190	188	145
99	55	54	55	74	23	12
45	55	56	45	155	145	156

To illustrate the four-step procedure above, we will work through how to compute the output pixel value at coordinate (1,1):

1. Extract the 3×3 image segment around the pixel at position (1,1). Remember that the top left pixel is at position (0,0). The pixel value at position (1,1) is **bolded** and the 3×3 segment is shown by the red border.

4	87	233	245	227	209	190
2	**59**	235	246	229	219	200
17	99	230	220	211	210	201
46	58	196	165	201	179	150
82	63	41	169	190	188	145
99	55	54	55	74	23	12
45	55	56	45	155	145	156

2. Multiply each pixel from this segment with the corresponding weight in the kernel

4	87	233
2	59	235
17	99	230

×

1/16	1/8	1/16
1/8	1/4	1/8
1/16	1/8	1/16

=

4/16	87/8	233/16
2/8	59/4	235/8
17/16	99/8	230/16

3. Sum each of the products from step 2

$$\frac{4}{16} + \frac{87}{8} + \frac{233}{16} + \frac{2}{8} + \frac{59}{4} + \frac{235}{8} + \frac{17}{16} + \frac{99}{8} + \frac{230}{16} = 97.875$$

4. Truncate the sum to an integer

```
int(97.875) = 97
```

The resultant pixel at location (1,1) in the output image would be 97. We would then repeat this process for the other pixels. The output pixel values for the entire 7×7 image is shown in the table below. We encourage you to try and compute the pixel value at location (2,1) by hand to make sure you have a sound understanding of the algorithm.

0	0	0	0	0	0	0
0	97	195	233	225	212	0
0	100	184	215	210	202	0
0	88	145	181	192	185	0
0	69	91	131	152	141	0
0	61	60	84	105	98	0
0	0	0	0	0	0	0

Why are all the pixels at the edge equal to 0?

At the edge of the image, we cannot extract a full 3×3 segment around the pixels because the 3×3 window would extend beyond the border of the image. So, for simplicity, we set the output to 0 for all pixels in edge regions where a full $N \times N$ window cannot be extracted.

In the next parts of the lab, you will write three functions to implement this kernel filtering process. We will begin by writing two helper functions to complete steps 1, 2, and 3 from the four-step procedure outlined above and finish by writing a function to perform these steps for all pixels in the image.[1]

Part 2.2 Dot Product

If we convert the kernel weights and image segments into vectors, steps 2 and 3 become a dot product operation from linear algebra. Here, you will write a function to perform this dot product operation.

The `dot_product` function accepts two tuples, each representing a vector, as inputs. You may assume the tuples are of equal length and only contain numerical values. The function should compute and return the dot product of the two vectors as a **float**.

Sample Test Case

Inputs		**Expected Output**
X	**Y**	
`(1, 1, 1)`	`(1, 2, 3)`	`6.0`

[1] If you're interested, check out this great interactive tool that shows you how output image pixels are computed using segments of the input image and the kernel: https://setosa.io/ev/image-kernels/.

Part 2.3 Extracting NxN Image Segments – extract_2D_img_segment function

The `extract_2D_img_segment` function will be used to complete step 1 of the four-step procedure by extracting and returning a $N \times N$ segment of pixels from a tuple of pixels representing an image. This function accepts the following input arguments:

- `img` – A tuple of grayscale image pixel values
- `w` – The width of the input image in pixels
- `h` – The height of the input image in pixels
- `centre_coordinate` – A two-element tuple containing the coordinate specifying the centre of the segment to extract
- `N` – Integer specifying the width and height of the segment to extract, N will always be a positive and odd integer

The function should extract the $N \times N$ segment of pixels centred around the `centre_coordinate` and **return the pixel values contained within the segment as a 1-dimensional tuple**. You may assume that the centre coordinate is **not** within an edge region where the full window is undefined.

Hints:

1. You will need to devise a way to compute the index of a pixel using it's coordinate and the width of the image.
2. Start by creating a function that extracts a single pixel from a desired location. Then extend this by adding a loop to extract pixels from the desired columns from a single row. Once you have this working correctly, extend your solution again by adding another (nested) loop to extract pixels from multiple rows.
3. Your final solution will likely contain a nested for loop. The outer loop should iterate over row numbers and the inner loop should iterate over column numbers. You can use these row and column numbers to calculate a pixel index.

Sample Testcases

For each testcase, assume that the `img` variable refers to the pixel values for the 7×7 image used in the example in part 2.1:

```
>>> img = (4, 87, 233, 245, 227, 209, 190,
           2, 59, 235, 246, 229, 219, 200,
           17, 99, 230, 220, 211, 210, 201,
           46, 58, 196, 165, 201, 179, 150,
           82, 63, 41, 169, 190, 188, 145,
           99, 55, 54, 55, 74, 23, 12,
           45, 55, 56, 45, 155, 145, 156)
```

Extract 3x3 window centred at (4,1)

```
>>> extract_2D_img_segment(img, 7, 7, (4,1), 3)
(245, 227, 209, 246, 229, 219, 220, 211, 210)
```

Extract 5x5 window centred at (2,2)

```
>>> extract_2D_img_segment(img, 7,7, (2,2), 5)
(4, 87, 233, 245, 227, 2, 59, 235, 246, 229, 17, 99, 230, 220, 211,
46, 58, 196, 165, 201, 82, 63, 41, 169, 190)
```

Part 2.4 Put it Together – kernel_filter function

The `kernel_filter` function takes the following as input parameters:

- `kernel` – A **nested tuple** defining the $N \times N$ two-dimensional kernel weights. Each element of the tuple is a row of kernel weights. N must be an odd integer. [2]
- `img` – A tuple of grayscale image pixel values
- `width` – The width of the input image
- `height` – The height of the input image

The function should **return a tuple of grayscale pixel values** representing the filtered image. The output image should be the same size (height and width) as the input. All pixels in the edge region of the output should be set to zero.

Hints:

1. As part of this function, you will need to determine the size of the edge region where a full $N \times N$ window cannot be extracted.
2. Consider using a nested for loop to iterate over the output image's pixel positions. The outer loop can iterate over the rows and the inner loop can iterate over the columns.

Testing Your Functions without Loading Images

To help you test and your code, we have provided a file, `lab7_gradescope_for_students.py` which will simulate the 5 Gradescope testcases on your computer. You need to save this file in the same folder as the code you are writing (i.e. `lab7.py`) and execute it. It will import your `lab7.py` and run the same tests as Gradescope. This program should give you the same behaviour as Gradescope and help you to track down any bugs in your functions.

Part 3 – Harris Corner Computations

There is no code to write for this part of the assignment. Instead, we will describe the output of the `harris_corners` function which you will be used as the input to the `non_maxima_suppression` function in the next part.

The Harris Corner Detector algorithm is an algorithm to infer the location of corners within an image. The algorithm takes a grayscale image as an input and computes a "corner strength" for

[2] If you're interested in learning more about image filtering and experimenting with different kernel filters, check out this tutorial: https://setosa.io/ev/image-kernels/

each pixel in the image. The algorithm then identifies the locations of all pixels with a corner strength greater than a specific threshold. These locations are then sorted according to the corner strength value from greatest to smallest. Our `harris_corners` function returns these sorted corner locations as a tuple.

However, this function often returns many pixel locations which are very close to each other (see all the overlapping red circles in Figure 4 below). Many of the locations returned are identifying the same corner. Ideally, want our corner detector to only identify one pixel location per corner. In the final part of this lab, you will implement a function that will limit the number of corners returned by the algorithm within subregions of the image.

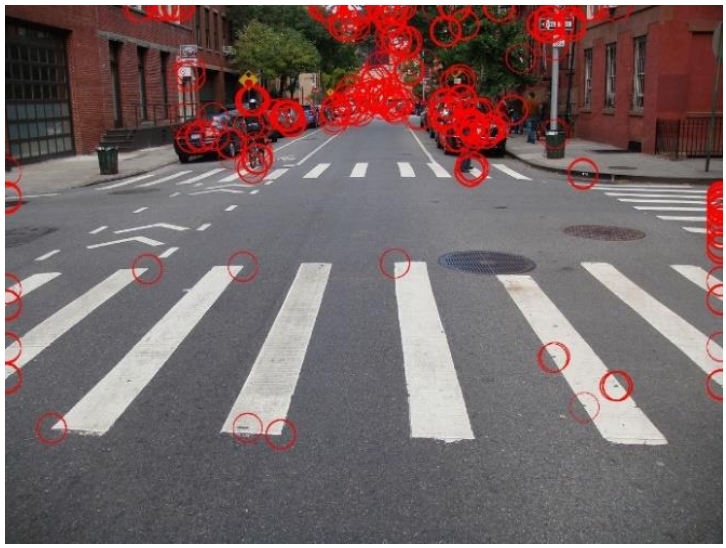


Figure 4. Output of Harris Corners without Non-Maxima Suppression. Each red circle indicates a detected corner.

Part 4 – Non-Maxima Suppression

The `non_maxima_suppression` function takes the following input parameters:

- `corners` – A tuple of corner locations, sorted from strongest to weakest, as returned by `harris_corners`
- `min_distance` – A float defining the minimum allowed distance between corners returned by this function

This function filters the list of corners to remove any corners that are within a specified distance to corner with a greater corner strength. This filtering is achieved using the following algorithm:

1) Initialize an empty list of unsuppressed corners *F*
2) For each potential corner *c* in the *corners* list:
   a) Calculate the Euclidean distances between *c* and all corners within *F*
   b) Add *c* to *F, unless* any of the Euclidean distances from 2a are less than *min_distance,*
3) Convert *F* to a tuple and return

The Euclidean distance between two pixels *i* and *j* with locations (x_i, y_i) and (x_j, y_j), can be computed as

$$\left(\left(x_i - x_j\right)^2 + \left(y_i - y_j\right)^2\right)^{\frac{1}{2}}$$

Let's work through an example to see how this works. Suppose our input list of potential corner coordinates is `((5,6), (15,6), (17,5))` and `min_distance = 10.`

In step 1, we create an empty list of unsuppressed corners

```
F = []
```

Then we start to iterate through all the corners of our input list. The first coordinate *c* is `(5,6)`. In step 2a, we need to calculate the distance between *i* and all the coordinates in *F* and check if any of these are less than `min_distance`. Since *F* is empty, there are no distances to calculate and therefore, there are no distances less than `min_distance`. So, we add *c* to *F*.

```
F = [(5,6)]
```

Now we return to step 2 and set *c* to the next coordinate `(15,6)` and then calculate the distances between *c* and all the points in *F.* The Euclidean distance between `(5,6)` and `(15,6)` is exactly 10. Since 10 is not less than `min_distance`, *c* is added to *F*.

```
F = [(5,6), (15,6)]
```

Now we repeat the process for the final coordinate `(17,5)`. The distances between `(17,5)` and the points within *F* are 12.04 and 2.24. Since 2.24 is less than `min_distance`, we do not add `(17,5)` to *F*. Now that we have evaluated every point, we convert *F* to a tuple and return.

Conclusion

That's it, you've used your programming superpowers to make a corner detector program. Congratulations! You have come a long way since your simple one-or-two-line programs in the first week of the course. Take a moment to reflect on everything you have learned about programming in the short amount of time since the beginning of term and what you have just been able to accomplish in this lab.